

Compiladores

Semântica e código intermediário

REALIZAR APÓS posição 13: Aula 11 - Geração de Código Intermediário | Três Endereços | Árvore de Sintaxe | Compiladores

Posições na playlist 12 a 13 | 120 minutos | 100 pontos

Curso principal: Compiladores |

<https://youtube.com/playlist?list=PLX6Nyaq0ebfhI396WIWN6WIBm-tp7vDtV&si=LoaU9IzLMuSVikgi>

Conteúdo verificável: AST; tipos; três endereços

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

AST e árvore concreta

Para $x=(a+2)*b$, desenhe AST e compare com árvore de derivação que inclui pontuação e não terminais. Quais informações devem ser preservadas para diagnóstico de erro e quais símbolos podem ser omitidos da AST?

Desenvolvimento / justificativa / esquema:

Regras de tipos

Tipos disponíveis int, float, bool; int pode converter para float, mas bool não participa de aritmética. $x:\text{int}$, $y:\text{float}$, $ok:\text{bool}$. Analise $x+y$, $ok+x$ e $x=y$ sob atribuição que não admite estreitamento implícito. Escreva regras de checagem e conversão da primeira expressão.

Desenvolvimento / justificativa / esquema:

Três endereços

Gere código de três endereços para $x = (a + b) * (c - d)$. Depois gere para `if(a < b) x = a; else x = b.`
Mostre temporários/rótulos e explique diferença entre temporário e variável fonte.

Desenvolvimento / justificativa / esquema:

Passagem semântica integradora

Escreva pseudocódigo que percorra AST de adição, número e identificador, devolvendo tipo e local do valor, e emitindo três endereços. Use regras da questão 2. Execute em $a+2.5$ com $a:\text{int}$ e mostre onde detectar $z+1$ se z não foi declarado.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - AST e árvore concreta (15 pontos)

AST: atribuição(x , multiplicação(adição(a , 2), b)). Parênteses e não terminais auxiliares podem desaparecer porque estrutura já preserva precedência; posições de origem devem ser associadas para diagnóstico. Árvore concreta acompanha produções/tokens, AST retém operações e relações relevantes. Não omitir uma operação que altera significado.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Regras de tipos (25 pontos)

$x+y$:float, converter x de int para float antes da soma. $ok+x$ é erro. $x=y$ é erro por float→int não implicitamente permitido. Regra aritmética aceita dois numéricos, resultado float se algum float, senão int. Atribuição exige igualdade ou ampliação int→float. Regras são as do enunciado, não suposições de outra linguagem.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Três endereços (25 pontos)

```
t1 = a + b
t2 = c - d
t3 = t1 * t2
x = t3
```

```
if a < b goto L1
x = b
goto L2
L1: x = a
L2:
```

Temporários representam resultados intermediários gerados pelo tradutor e não precisam existir no código fonte. Cada instrução tem operação elementar; rótulos explicitam fluxo. Aceitar outra disposição de saltos equivalente.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Passagem semântica integradora (35 pontos)

```
traduzir(nó):  
  número: retornar (tipo_literal, literal)  
  identificador: buscar símbolo; retornar (tipo, nome)  
  adição:  
    (t1,p1)=traduzir(esquerdo); (t2,p2)=traduzir(direito)  
    se algum tipo não numérico: erro  
    t = float se algum é float, senão int  
    converter operandos int para float quando necessário  
    r = novo_temporario(); emitir r = p1 + p2  
    retornar (t,r)
```

a+2.5 emite t1=int_to_float a; t2=t1+2.5, resultado float. z+1 falha na busca de z, antes de emitir adição válida. O percurso deve evitar gerar resultado aparentemente correto após erro e ligar diagnóstico à posição do nó.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.